



Improving Access to Meeting Minutes Knowledge Using Retrieval-Augmented Generation and Vector Databases

T2000

For the Semester Project for the

Bachelor of Science

of the degree program New Study Informatik

at the Baden-Württemberg Cooperative State University

from

Liam Thomson

September 2025

Processing period	26 Weeks
Matrikulation number, Course	3248571, DHBW-TINFO23
Training Company	Accenture, Kronberg im Taunus
Supervisor	Piotr Preuss
Examiner	Dr. Prof. Kay Berkling

Confidentiality Notice

The present T2000 thesis titled *Improving Access to Meeting Minutes Knowledge Using Retrieval-Augmented Generation and Vector Databases* is classified as confidential and is submitted solely for examination purposes in the New Study Informatik program at the Baden-Württemberg Cooperative State University at the submission location. Any inspection or publication – including partial excerpts of the thesis – requires prior approval from Accenture.

Declaration

I hereby declare under oath:

1. that I have completed my T2000 mit dem Thema on the topic *Improving Access to Meeting Minutes Knowledge Using Retrieval-Augmented Generation and Vector Databases* without external assistance;
2. that I have indicated the adoption of literal quotations from the literature and the use of the ideas of other authors at the corresponding points within the work;
3. that I have not submitted my T2000 for any other examination;

I am aware that a false declaration will have legal consequences.

Baden-Württemberg Cooperative State University, September 2025

Liam Thomson

Abstract

Meeting minutes are an essential resource for capturing decisions, discussions, and knowledge within organizations. However, when stored across multiple platforms such as Confluence, accessing relevant information becomes time-consuming and inefficient. Traditional keyword-based search often fails to capture semantic meaning, leading to incomplete or irrelevant results and limiting the practical value of meeting records.

This paper presents the design and implementation of a Retrieval-Augmented Generation (RAG) system that addresses these challenges by combining semantic vector search with large language models. The proposed solution preprocesses and normalizes meeting minutes, generates embeddings using a transformer-based model, and stores them in a local Qdrant vector database to ensure data privacy. A retrieval pipeline then selects semantically relevant chunks based on user queries and metadata filters, which are passed to an LLM to generate context-aware summaries and answers.

The system was evaluated using realistic queries with and without metadata filtering. Results show that semantic retrieval significantly improves relevance compared to keyword search, while metadata filters consistently ensure high precision for date-specific and domain-focused queries. Although inference latency was a limitation due to local hardware constraints, the overall performance demonstrated the feasibility and value of applying RAG to meeting minutes concluding that RAG-based retrieval can meaningfully improve access to organizational knowledge.

Table of Contents

1 Introduction **1**
- 1.1 Motivation . . . 1
- 1.2 Problem Statement and Research Questions . . . 1
- 1.3 Objectives and Success Criteria . . . 2
- 1.4 Scope and Assumptions . . . 3
- 1.5 Contributions . . . 3
- 1.6 Structure of the Paper . . . 4

2 Related Work & Background **5**
- 2.1 Enterprise Information Retrieval & Knowledge Silos . . . 5
- 2.2 Keyword Search Limits vs. Semantic Search . . . 5
- 2.3 Vector Search Fundamentals (Embeddings, Similarity, ANN) . . . 6
- 2.4 Embedding Models for Sentence Semantics . . . 6
- 2.5 Vector Databases (HNSW/IVF, Payload Filters): Qdrant Overview . . . 7
- 2.6 RAG Architectures, Pitfalls, Hallucinations . . . 7
- 2.7 Prior Art on Meeting Notes Search & Enterprise QA . . . 7

3 Requirements & Problem Analysis **9**
- 3.1 Stakeholders & User Stories . . . 9
- 3.2 Functional Requirements . . . 11
- 3.3 Non-Functional Requirements . . . 11
- 3.4 Evaluation Goals & Hypotheses . . . 12

4 System Architecture & Design **13**
- 4.1 High-Level Architecture . . . 13

5 Data Acquisition & Preprocessing **15**
- 5.1 Source Description . . . 15
- 5.2 Cleaning & Normalization . . . 15
- 5.3 Chunking Strategy . . . 16
- 5.4 Metadata Schema & Date Normalization . . . 16
- 5.5 Ground-Truth Creation for Evaluation . . . 17

6 Embeddings & Vector Indexing **18**
6.1 Model Selection . . . 18
6.2 Embedding Configuration & Batching . . . 18
6.3 Qdrant Setup . . . 19
6.4 Payload Design for Filters . . . 20
6.5 Scalability Considerations . . . 21

7 RAG Pipeline & Querying **22**
7.1 Retriever . . . 22
7.2 Filtering . . . 23
7.3 Prompt Design for Meeting-Minutes QA . . . 24
7.4 LLM Serving with Llama.cpp . . . 25
7.5 LLM Integration in Code . . . 25

8 Experimental Setup **27**
8.1 Hardware & Environment . . . 27
8.2 Software Stack . . . 27
8.3 Dataset . . . 28
8.4 Metrics & Instruments . . . 28

9 Results **30**
9.1 Retrieval & RAG Performance . . . 30
9.2 Answer Quality . . . 31
9.3 Case Studies and Error Analysis . . . 31

10 Discussion **33**

11 Conclusion & Future Work **34**
11.1 Conclusion . . . 34
11.2 Future Work . . . 35

Figures **i**

Tables **ii**

Listings **iii**

References **iv**

Abbreviations **v**

Glossary **vi**

1 Introduction

1.1 Motivation

Meeting minutes are a critical artifact in organizational knowledge management. They capture discussions, decisions, and action items that shape project trajectories and strategic directions. However, in many organizations—including the one motivating this study—minutes are dispersed across different repositories, often in Confluence spaces or shared drives. This fragmentation makes retrieval difficult and time-consuming.

Traditional keyword search engines, while useful for exact matches, often fail to surface semantically relevant content. For example, searching for "testing process" may not retrieve records that describe "validation procedures" or "QA steps," despite their conceptual similarity. Such limitations lead to inefficiencies, duplicated work, and missed insights.

In environments where rapid decision-making is essential, these barriers translate into measurable productivity losses. Employees spend significant time searching for information rather than acting on it, diminishing the strategic value of recorded knowledge. A more intelligent retrieval mechanism is therefore necessary to unlock the potential of meeting archives as an actionable knowledge base.

1.2 Problem Statement and Research Questions

The central problem addressed in this paper is the inadequacy of keyword-based search systems for retrieving relevant meeting minutes. Current approaches ignore semantic relationships between terms, lack contextual understanding, and often present users with irrelevant or incomplete results.

This leads to the following research questions:

- **RQ1:** How can meeting minutes be preprocessed, vectorized, and stored to enable efficient semantic retrieval?
- **RQ2:** To what extent can a RAG[1] (Retrieval-Augmented Generation) pipeline improve answer quality and retrieval speed compared to traditional keyword search?
- **RQ3:** How can Metadata filters (e.g., meeting type, date range) enhance precision and contextual relevance in search results?

1.3 Objectives and Success Criteria

The main objective of this study is to design and evaluate a vector database–driven RAG solution for retrieving and synthesizing knowledge from meeting minutes. This entails building a reproducible pipeline that ingests unstructured text, enriches it with metadata, generates semantic embeddings, and leverages a LLM[2] to deliver context-aware answers.

The success of this project is defined by the following criteria:

1. **Technical feasibility:** The system should successfully ingest and vectorize meeting minutes into a searchable database.
2. **Retrieval relevance:** Search results should show measurable relevance.
3. **Answer quality:** LLMs-generated responses should be accurate, concise, and grounded in source documents.
4. **Efficiency:** Average search and answer time should be within seconds.
5. **Practical viability:** The system should be deployable locally (on-premises) to satisfy privacy requirements.

[1] Retrieval-Augmented Generation
[2] Large Language Model

1.4 Scope and Assumptions

This research focuses on meeting minutes stored as plain-text documents exported from Confluence. Only text-based minutes are considered; audio transcripts and multimedia sources fall outside the current scope. Metadata such as meeting type and date are assumed to be available through accompanying `.json` files.

The system is designed for on-premises deployment, reflecting privacy and compliance requirements in enterprise contexts. Cloud-based vector stores and LLM are not used to avoid data exposure and also due to limited resources. Instead, embeddings are generated locally using transformer-based models and stored in a local Vector Database instance.

It is assumed that users are familiar with basic enterprise search tools and can interact with a simple query interface. Evaluation is limited to retrieval and generation quality; downstream integration with project management systems is beyond scope.

1.5 Contributions

This paper makes the following contributions:

1. A preprocessing and vectorization pipeline for meeting minutes that integrates text chunking, metadata normalization, and embedding generation.
2. A RAG framework that combines a vector database with an LLM for semantic search and natural language answers.
3. A filterable retrieval mechanism leveraging metadata fields such as meeting type and date, enabling more precise and context-sensitive queries.
4. An evaluation of the RAG-based retrieval in terms of relevance, efficiency, and answer quality

1.6 Structure of the Paper

The remainder of this paper is organized as follows:

- **Section 2** reviews related work in enterprise information retrieval, vector databases, Embedding models, and RAG pipelines.
- **Section 3** analyzes the requirements and formulates evaluation goals.
- **Section 4** presents the system architecture and design, including data modeling and security considerations.
- **Section 5** describes the data preprocessing and metadata normalization procedures.
- **Section 6** explains the embedding and indexing process.
- **Section 7** details the RAG pipeline.
- **Section 8** outlines the experimental setup, including datasets,, and evaluation metrics.
- **Section 9** reports and analyzes results.
- **Section 10** discusses findings and implications.
- **Section 11** concludes the paper and identifies directions for future work.

2 Related Work & Background

2.1 Enterprise Information Retrieval & Knowledge Silos

Enterprise Information Retrieval is concerned with enabling employees to locate relevant information across organizational systems (**?**). Despite advances in search technologies, many organizations still suffer from knowledge silos: information is scattered across document repositories, intranets, and knowledge bases, often without a unified indexing strategy. As a result, employees face high Information Retrieval costs, leading to duplicated work and delayed decision-making.

Meeting minutes represent a particularly challenging case, as they are usually maintained in semi-structured text across platforms such as Confluence, SharePoint, or local file systems. Unlike structured databases, meeting notes rely heavily on natural language, contextual cues, and implicit references, which traditional enterprise search solutions struggle to process effectively.

2.2 Keyword Search Limits vs. Semantic Search

Traditional search approaches such as Term Frequency–Inverse Document Frequency and Best Matching 25 are well-established in the Information Retrieval community. These methods calculate document relevance based on keyword overlap and frequency statistics. While efficient and scalable, they lack an understanding of linguistic phenomena such as synonymy, paraphrasing, and context.

For example, when a user queries "customer onboarding," a keyword-based system may miss documents that describe "client initiation" or "account setup." This lexical

mismatch problem is a well-known limitation in keyword search. Semantic search, by contrast, leverages distributed representations of words and sentences to capture underlying meaning, thereby retrieving conceptually similar content even when the exact keywords differ.

2.3 Vector Search Fundamentals (Embeddings, Similarity, ANN)

Vector search underpins semantic retrieval by mapping documents and queries into dense vector representations, also known as embeddings. These embeddings are placed in a high-dimensional vector space where semantic similarity is measured using distance metrics such as cosine similarity or Euclidean distance.

Efficient retrieval at scale requires Approximate Nearest Neighbor (ANN) algorithms, as exact search becomes computationally expensive in high dimensions. ANN techniques such as Hierarchical Navigable Small Worlds (HNSW) and Inverted File Indexes (IVF) offer sub-linear retrieval times while maintaining acceptable recall. Vector search thus enables semantic retrieval in large enterprise settings where both query volume and corpus size can be substantial.

2.4 Embedding Models for Sentence Semantics

The quality of vector search depends heavily on the embedding model used. Early embedding methods such as Word2Vec and GloVe captured semantic meaning at the word level, but lacked contextual awareness. Transformer-based models, such as BERT and its successors, introduced contextualized embeddings by considering entire sentences.

Sentence embedding models such as Sentence-BERT and `all-MiniLM-L6-v2` provide lightweight, high-performing sentence-level representations optimized for similarity tasks. The `all-MiniLM-L6-v2` model, in particular, balances performance and computational efficiency, making it suitable for enterprise-scale applications where embeddings need to be generated on local infrastructure. These models form the backbone of modern semantic search systems, including the one implemented in this work.

2.5 Vector Databases (HNSW/IVF, Payload Filters): Qdrant Overview

Storing and retrieving high-dimensional embeddings requires specialized vector databases. Popular solutions include FAISS, Weaviate, Pinecone, and Qdrant. These systems provide scalable indexing structures, ANN algorithms, and metadata-aware filtering.

Qdrant, the vector database used in this project, supports HNSW-based indexing and provides flexible payload filters for structured metadata (e.g., date, meeting type). This makes it well-suited for enterprise meeting minutes, where queries often need to be scoped to specific time ranges or project categories. Moreover, Qdrant supports on-premises deployment, aligning with privacy and compliance requirements that are critical in many enterprise environments.

2.6 RAG Architectures, Pitfalls, Hallucinations

RAG has emerged as a powerful paradigm that combines information retrieval with large-scale generative models. In a RAG pipeline, a retriever identifies relevant documents, and an LLM conditions on this evidence to generate an answer. This architecture mitigates the limitations of closed-book models, which may hallucinate facts when operating without context.

However, RAG introduces its own challenges. If the retriever fails to surface the correct documents, the generator produces answers based on incomplete or irrelevant context. Moreover, generative models may still hallucinate details, presenting plausible but inaccurate information. To mitigate these risks, system design must emphasize retriever quality, evidence grounding, and transparency of source citations.

2.7 Prior Art on Meeting Notes Search & Enterprise QA

Research on meeting notes retrieval remains relatively limited compared to broader document search. Prior systems often rely on keyword-based enterprise search engines

or manually curated knowledge bases. Recent advances in enterprise QA have introduced neural retrieval systems that integrate structured and unstructured sources, but these are rarely applied directly to meeting archives.

Industry solutions, such as Microsoft Viva Topics or Atlassian's in-product search, provide surface-level retrieval but remain limited by keyword or entity-based indexing. To our knowledge, few deployed systems combine vector search, metadata filtering, and RAG in the context of meeting minutes. This gap motivates our work, which applies modern semantic retrieval methods to enhance access to meeting records, while addressing enterprise-specific constraints such as on-premises deployment and data privacy.

3 Requirements & Problem Analysis

3.1 Stakeholders & User Stories

Meeting minutes in the organization are stored in different places, including chat transcripts from Microsoft Teams (often generated with Copilot), Confluence pages, or local files. Regardless of their origin, these minutes are semi-structured and often lack consistent metadata, which complicates retrieval and reuse. The proposed system addresses the needs of three key stakeholder groups. Understanding their perspectives ensures that the solution is both technically sound and practically useful.

Requirements Traceability Matrix

The following table summarizes the relationships between stakeholders, their needs, and the functional requirements derived for the system:

Stakeholder	Need / User Story	Mapped Functional Requirement(s)
Project Manager	Quickly retrieve decisions from previous meetings to support informed project choices.	FR2: Semantic search; FR4: Retrieval-Augmented Generation; FR5: Source traceability
Engineer	Search for technical discussions (e.g., testing, architecture, bug fixes) to avoid duplication and align with established practices.	FR1: Document ingestion; FR2: Semantic search; FR3: Filtered retrieval
Consultant / Associate	Access concise answers to project-related questions and locate relevant meeting decisions efficiently to support daily client work.	FR2: Semantic search; FR4: Retrieval-Augmented Generation; FR5: Source traceability

Table 3.1: Requirements Traceability Matrix: Linking stakeholders to functional requirements.

Project Managers

Project Managers are primarily concerned with rapid decision-making and traceability of discussions. **User story:** As a Project Manager, I want to quickly retrieve decisions from previous meetings, so that I can make informed project choices without having to manually read through extensive meeting minutes.

Engineers

Engineers require technical context, such as decisions made about testing, architecture, or bug fixes, to avoid duplication of work. **User story:** As an Engineer, I want to search for previous discussions on system testing, so that I can reuse knowledge and align my work with established practices.

Consultants / Associates

Consultants and associates often work across multiple projects and require quick access to relevant discussions without needing to know where the minutes were stored. **User story:** As a Consultant, I want to receive summarized answers from past meetings, so that I can efficiently support client-facing work and stay aligned with project progress.

3.2 Functional Requirements

The core system capabilities are defined as follows:

1. **Document ingestion:** The system must ingest meeting minutes from text-based sources (e.g., Confluence exports) and normalize metadata such as meeting type and date.
2. **Semantic search:** Users must be able to issue natural language queries and retrieve relevant content based on semantic similarity, not just keyword matching.
3. **Filtered retrieval:** Users should refine searches by metadata, e.g., restricting results by meeting type or within a specified date range.
4. **Retrieval-Augmented Generation:** The system must provide contextualized answers by combining retrieved passages with an LLM, returning responses grounded in the original documents.
5. **Source traceability:** Every generated answer must include references or metadata that point back to the originating meeting notes for verification.

3.3 Non-Functional Requirements

Beyond functionality, the system must meet several quality and operational constraints:

- **Latency:** Retrieval and generation should complete within a few seconds, ensuring usability in daily workflows.

- **Accuracy:** The system should maximize retrieval relevance and minimize hallucinations in generated answers, as measured by relevance metrics and human evaluation.
- **Privacy:** Since meeting notes may contain sensitive information, the entire pipeline must operate on-premises, without reliance on cloud services. For the same reason, actual LLM output examples are omitted in this report, as real project meeting minutes were used during evaluation.
- **Scalability:** The database must handle growing volumes of meeting minutes over time, without significant performance degradation.

3.4 Evaluation Goals & Hypotheses

To validate the effectiveness of the solution, the following evaluation goals are established:

- Measure the relevance of retrieved results compared to keyword-based search using metrics such as **nDCG@5!**[1] and Recall@5.
- Assess the quality of generated answers through human evaluation.
- Measure the efficiency of the system (retrieval and response times).

The evaluation is guided by the following primary hypothesis:

> **H1:** A RAG-based retrieval system can provide accurate, context-enriched answers to natural language prompts.

Secondary hypothesis:

- **H2:** Metadata filtering (e.g., by date or meeting type) increases the precision of retrieval without reducing recall significantly.

[1] **nDCG@5!**

4 System Architecture & Design

4.1 High-Level Architecture

The system transforms raw meeting minutes into semantically searchable knowledge. Two main components make up this architecture: the *Document Vectorizer*, which prepares and indexes the documents, and the *RAG Query Pipeline*, which retrieves relevant content and generates answers. Figures 4.1 and 4.2 illustrate these pipelines.

Document Vectorizer

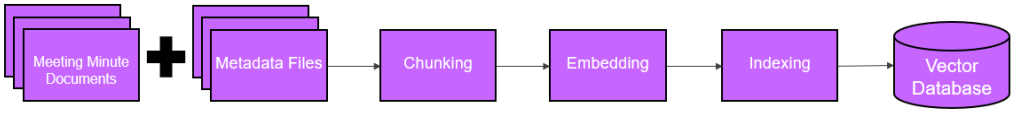


Abbildung 4.1: Document Vectorizer: Documents & Metadata → Chunking → Embedding → Indexing → Vector Database

The Document Vectorizer processes meeting minutes and prepares them for efficient semantic search. It consists of four main steps:

- **Meeting Minute Documents & Metadata:** Raw text documents are combined with optional metadata (e.g., meeting type, date), allowing filtering for later retrieval.
- **Chunking:** Long documents are segmented into smaller, coherent text units. This ensures fine-grained retrieval and avoids mixing unrelated topics.
- **Embedding:** Each chunk is converted into a numerical vector representation that captures semantic meaning, enabling similarity-based retrieval.

- **Indexing:** The vectors and their metadata are stored in a vector database, optimized for fast nearest-neighbour search and filterable queries.

RAG Query Pipeline

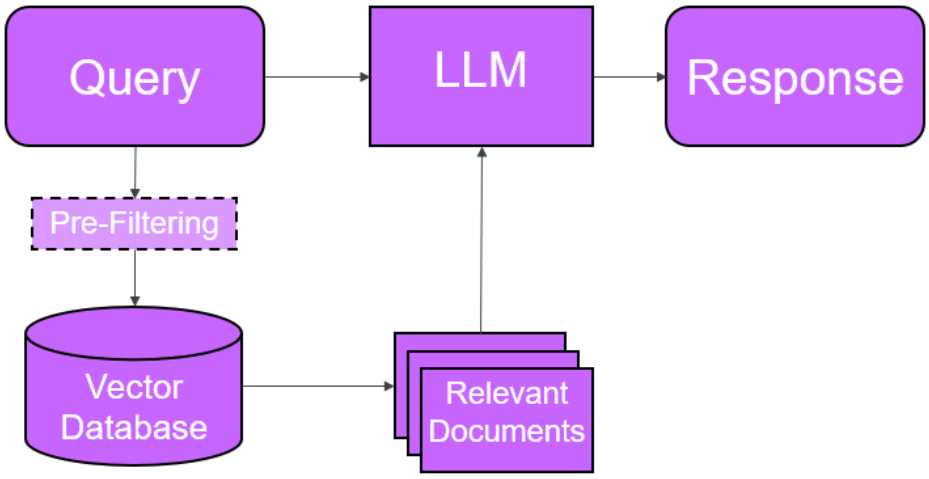


Abbildung 4.2: RAG Query Pipeline: Query → Pre-Filtering → Vector Database Retrieval → Relevant Documents → LLM → Response

The RAG Query Pipeline transforms a user's natural language query into a grounded answer. It works in six main steps:

- **Query:** The user creates a query (e.g., "What was discussed about E-net testing?").
- **Pre-Filtering:** The user can optionally apply filters (e.g., meeting type or date range) to improve retrieval of relevant documents.
- **Vector Database:** The system retrieves the most semantically relevant document chunks based on similarity search in the vector database using the query and filter parameters.
- **Relevant Documents:** The top 5 most relevant documents are retrieved from the Vector Database and passed to the LLM.
- **LLM:** The retrieved documents and the original query are passed to the LLM, which generates a coherent response grounded in the underlying documents.
- **Response:** The user can read the response from the LLM and see the relevant documents passed to it in order.

5 Data Acquisition & Preprocessing

5.1 Source Description

The meeting minutes used in this project originate from multiple sources within the organization. These include:

- **Microsoft Teams chats:** Often generated automatically by Copilot during or after a meeting.
- **Confluence pages:** Exported from project spaces and stored as plain-text files.
- **Local files:** Manually written or shared documents, typically in text format.

For uniformity, all sources were exported into the directory `Docs/Fresh Docs`, which serves as the ingestion point for preprocessing. After successful vectorization, documents are moved to `Docs/Vectorized Docs` to prevent reprocessing.

5.2 Cleaning & Normalization

To ensure consistency, all text is normalized to **UTF-8** encoding. Common issues such as non-standard characters, line breaks from copy-paste, and inconsistent whitespace were addressed.

Additionally, boilerplate text (e.g., "Meeting notes generated by Copilot" headers) was stripped, as it does not contribute semantic value and risks polluting the embeddings. Headings and timestamps were retained, since they provide contextual cues (e.g., agenda items or meeting sequences) that can improve retrieval quality.

5.3 Chunking Strategy

Meeting minutes vary in length, from short daily standups to multi-page strategy sessions. To handle this, documents were split into overlapping chunks using the `RecursiveCharacterTextSplitter` from LangChain.

- **Chunk size:** 500 characters
- **Overlap:** 100 characters

This strategy balances two goals:

1. Preventing loss of context — overlap ensures that sentences cut at chunk boundaries are still semantically complete in at least one chunk.
2. Maintaining efficiency — smaller chunks enable finer-grained retrieval and reduce the risk of exceeding the context window of the LLM.

In practice, this produced an average of 6–11 chunks per meeting document, which was manageable both in storage and retrieval latency.

5.4 Metadata Schema & Date Normalization

Each document chunk is paired with a metadata file to enable filtering and traceability. The schema is shown in Table 5.1.

Field	Type	Description
date	Float (Unix timestamp, UTC)	Normalized date for range filtering
Meeting_type	String	Category of meeting (e.g., Brownbag Session, Sprint Planning)

Table 5.1: Metadata schema for meeting minutes chunks.

Date normalization is performed by the helper function `to_ts_utc`, which accepts multiple formats:

- **DD/MM/YYYY strings:** e.g., `"30/07/2025"`
- **Numeric strings:** e.g., `"1753999200"` or `"1753999200.0"`

Invalid dates are logged with a warning and removed from the metadata. This approach ensures robustness, given the variability of exported files.

5.5 Ground-Truth Creation for Evaluation

To evaluate the system, a small ground-truth dataset was manually constructed. A set of **queries** was defined, reflecting realistic user needs (e.g., "What decisions were made about e-net testing last week?"). This dataset provides the basis for computing metrics such as Recall@5 and nDCG[1], as well as for qualitative analysis of generated answers.

Implementation Anchors

The preprocessing pipeline is implemented in `DocVectorizer.py`. Key steps include:

- Loading documents with the LangChain `TextLoader`.
- Merging metadata from `.json` files with the loaded text.
- Splitting documents into chunks using the RecursiveCharacterTextSplitter.
- Normalizing dates via the `to_ts_utc` function.
- Verifying metadata integrity and discarding invalid entries.
- Moving successfully processed files from `Docs/Fresh Docs` to `Docs/Vectorized Docs`.

This design helps to mitigate accidental re-vectorization of documents, preventing duplication and improving efficiency. The chosen chunk size and overlap represent a deliberate trade-off between context preservation and retrieval efficiency.

[1] Normalized Discounted Cumulative Gain

6 Embeddings & Vector Indexing

6.1 Model Selection

The choice of embedding model is critical for achieving semantically meaningful retrieval. This project employs the HuggingFace model `all-MiniLM-L6-v2`.

This model was selected for the following reasons:

- **Accuracy:** Provides strong performance on semantic similarity benchmarks despite being significantly smaller than transformer-based alternatives like BERT or RoBERTa.
- **Efficiency:** The compact size enables fast inference, which is decent at processing large numbers of meeting minutes locally.
- **Compatibility:** Fully integrated into the LangChain framework, making it easy to use for document embeddings and query encoding.

Alternative models such as `multi-qa-mpnet-base-dot-v1` could provide higher accuracy, but at substantially increased computational cost. Given the requirement for on-premises deployment and responsiveness, `all-MiniLM-L6-v2` has a balance between speed and accuracy.

6.2 Embedding Configuration & Batching

Each document chunk produced during preprocessing is transformed into a dense vector of dimension 384. This size reflects the representational capacity of the chosen MiniLM model.

Embedding generation is performed in batches to optimize throughput. While the LangChain interface abstracts batch size configuration, empirical testing showed that batching improves GPU utilization and reduces embedding time per document. The trade-off lies in memory usage, which must be tuned according to hardware constraints.

To maintain reproducibility and consistency, the following parameters are fixed:

- **Model:** `all-MiniLM-L6-v2`
- **Vector dimensionality:** 384
- **Distance metric:** Cosine similarity

6.3 Qdrant Setup

The embeddings and metadata are stored in a local Qdrant[1] instance.

The collection lifecycle is managed directly in `DocVectorizer.py`, as shown in Listing 6.1. If the collection does not exist, it is created; otherwise, the existing collection is reused.

```
client = QdrantClient(path=QDRANT_PATH)

existing = [col.name for col in client.get_collections().
    collections]
if COLLECTION_NAME not in existing:
    client.recreate_collection(
        collection_name=COLLECTION_NAME,
        vectors_config=VectorParams(size=384, distance=
            Distance.COSINE),
    )
    print("Created new Qdrant collection.")
else:
    print("Using existing Qdrant collection.")
```

Listing 6.1: Qdrant collection setup in DocVectorizer.py

[1] Qdrant Vector Database

6.4 Payload Design for Filters

Each chunk inherits file-level metadata during preprocessing and is persisted 1:1 as the Qdrant *payload* alongside its embedding. In practice, the `Document.metadata` dict becomes the stored payload, giving every vector a stable `doc_id`, `chunk_idx`, `date` (UTC epoch), `meeting_type`, and optional `tags`. At query time, filters are executed server-side in Qdrant *before* ANN ranking, so only vectors matching the payload constraints (e.g., meeting type + date range) are considered for similarity search, improving precision without extra post-processing.

```
from langchain_core.documents import Document
from langchain_qdrant import QdrantVectorStore

vectorstore = QdrantVectorStore(
    client=client,
    collection_name="document_vectors",
    embeddings=embeddings,
)

# During indexing: metadata on each chunk becomes Qdrant
    payload
chunk = Document(
    page_content="...text chunk...",
    metadata={
        "doc_id": "2025-08-05_daily_notes",
        "chunk_idx": 3,
        "source": "Daily_05-08-2025.txt",
        "date": 1754332800.0,  # UTC epoch (Chapter 5
            normalization)
        "meeting_type": "Brownbag",
        "tags": ["testing", "e-net"],
    },
)
vectorstore.add_documents([chunk])  # payload is stored with
    the vector
```

Listing 6.2: Linking metadata to Qdrant payload and filtering at retrieval

6.5 Scalability Considerations

While the prototype operates on a moderate number of meeting minutes, the system design anticipates growth. Qdrant provides several features relevant for scalability:

- **Sharding:** Collections can be distributed across nodes (independent server instances that share the workload) to handle larger datasets.
- **Snapshots:** Regular snapshots (point-in-time backups of the entire collection state) ensure persistence and enable recovery in case of system failure.
- **Compaction:** Automatic compaction (the process of merging and reorganizing stored data) optimizes storage usage as vectors are updated or deleted.

Future scaling scenarios include handling thousands of meeting minutes per month, requiring careful monitoring of indexing latency, storage overhead, and retrieval speed.

Implementation Anchors

The embedding and indexing pipeline is implemented in `DocVectorizer.py`. Key aspects include:

- Initializing the HuggingFace embeddings object.
- Creating or reusing a Qdrant collection with cosine distance and 384-dimension vectors.
- Attaching metadata payloads for each chunk before ingestion.
- Adding the chunks to the vector store for subsequent retrieval.

7 RAG Pipeline & Querying

7.1 Retriever

The retriever is the component responsible for fetching semantically relevant document chunks in response to a user query. In this project, the retriever is built on top of the `QdrantVectorStore` provided by LangChain.

Two main interfaces are used:

- `similarity_search`: Returns the top-k most similar documents to a query embedding.
- `as_retriever`: Wraps similarity search into a retriever API that integrates seamlessly with the `RetrievalQA` chain.

Hyperparameters include:

- **k:** The number of top results retrieved (default 5). This is often referred to as "top-k" retrieval, meaning the k highest-scoring document chunks are returned in ranked order. Increasing k raises recall but may reduce precision.
- **score_threshold:** An optional cutoff ensuring that only documents above a similarity threshold are returned.

```
retriever = vectorstore.as_retriever(
    search_kwargs={"k": 5}  # top-5 most similar results
)
```

Listing 7.1: Defining the retriever in Python

7.2 Filtering

Filtering constrains retrieval to relevant subsets of the indexed corpus using metadata stored as Qdrant payloads (cf. Chapter 6). In `RagQuery.py`, filters are optional and controlled by three toggles: `USE_FILTERS`, `USE_MEETING_TYPE_FILTER`, and `USE_DATE_RANGE_FILTER`. When enabled, the code builds a Qdrant-compatible filter dictionary that is passed through `search_kwargs` to the retriever. Meeting types are matched by exact value on the `Meeting_type` payload key, while date windows are expressed as a UTC epoch range computed from DD/MM/YYYY via `to_timestamp_utc`.

```
# === RAG Filter Toggles ===
USE_FILTERS = True
USE_MEETING_TYPE_FILTER = True
USE_DATE_RANGE_FILTER = True

# Filter Values
MEETING_TYPE = "Brownbag Session"
DATE_FROM = "30/07/2025"
DATE_TO = "07/08/2025"

# Convert DD/MM/YYYY to UTC timestamp
def to_timestamp_utc(date_str: str) -> float:
    return datetime.strptime(date_str, "%d/%m/%Y")\
        .replace(tzinfo=timezone.utc).timestamp()

# Filter
payload_filter = None
if USE_FILTERS:
    must_filters = []

    if USE_MEETING_TYPE_FILTER and MEETING_TYPE:
        must_filters.append({
            "key": "Meeting_type",
            "match": {"value": MEETING_TYPE}
        })
```

```
    if USE_DATE_RANGE_FILTER and DATE_FROM and DATE_TO:
        must_filters.append({
            "range": {
                "key": "date",
                "gte": to_timestamp_utc(DATE_FROM),
                "lte": to_timestamp_utc(DATE_TO),
            }
        })

    if must_filters:
        payload_filter = {"must": must_filters}

print("\n[DEBUG] Payload filter about to be used:")
print(payload_filter if payload_filter else "(no filters)")

retriever = vectorstore.as_retriever(
    search_kwargs={"filter": payload_filter} if payload_filter
        else {}
)
```

Listing 7.2: Applying metadata filters in RagQuery.py

When no filters are enabled, the system falls back to global similarity search; when active, only chunks matching `Meeting_type` and/or the specified `date` window are eligible for retrieval before ANN scoring.

7.3 Prompt Design for Meeting-Minutes QA

The quality of responses from the LLM depends strongly on prompt design. In this project, prompts are kept short, precise, and tailored to the meeting-minutes context to minimize ambiguity and ensure reliable outputs.

- **Conciseness:** Prompts and responses are formulated to be direct, avoiding unnecessary elaboration.

- **Relevance:** Prompts must be relevant to the context of the Meeting Minutes

Example prompt: *"What was discussed about e net testing in the dailies?"*

7.4 LLM Serving with Llama.cpp

The generative component is served locally using the LlamaCpp[1] library. The model configuration are the following:

- **Model file:** `llama-2-7b-chat-hf-q4_k_m.gguf`
- **Context window (n_ctx):** 2048 tokens
- **GPU layers:** 35 layers offloaded to GPU where available
- **Temperature:** 0.3, ensuring deterministic and concise outputs

7.5 LLM Integration in Code

The LLM is integrated into the RAG pipeline using the `LlamaCpp` class from the LangChain community package. This component is initialized with the locally stored quantized `llama-2-7b-chat-hf-q4_k_m.gguf` model.

Key configuration parameters include:

- **Model path:** Specifies the location of the quantized model file.
- **Context window (`n_ctx`):** Defines the maximum number of tokens the model can consider in a single generation.
- **GPU layers:** Controls how many transformer layers are offloaded to GPU to accelerate inference.
- **Temperature:** Adjusts the determinism of the generated text; a lower value (0.3) ensures concise and focused answers.

[1] Llama.cpp Library for running LLaMA models locally

The instantiated LLM object is then passed into the `RetrievalQA` chain, which combines the retrieved chunks with the user query to produce the final grounded response.

```
from langchain_community.llms import LlamaCpp
from langchain.chains import RetrievalQA

# Load quantized Llama 2 model
llm = LlamaCpp(
    model_path=MODEL_PATH,
    n_ctx=2048,
    n_gpu_layers=35,
    temperature=0.3,
    verbose=False,
)

# Combine retriever and LLM in a RetrievalQA chain
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",
    retriever=retriever,
    return_source_documents=True
)
```

Listing 7.3: Initializing and integrating the LlamaCpp model

Implementation Anchors

The RAG pipeline is implemented in `RagQuery.py`. Key aspects include:

- Initializing the Qdrant vector store with HuggingFace embeddings.
- Constructing payload filters for meeting type and date ranges.
- Configuring the retriever and connecting it to the `RetrievalQA` chain.
- Integrating the locally served LLM via `LlamaCpp` (cf. Section 7.3).
- Printing debug information to validate retrieval, filtering, and grounding.

8 Experimental Setup

8.1 Hardware & Environment

All experiments were conducted in a local, on-premises environment to comply with organizational privacy requirements.

- **CPU:** 11th Gen Intel(R) Core(TM) i7-1185G7 @ 3.00GHz
- **GPU:** Intel(R) Iris(R) Xe Graphics
- **RAM:** 32 GB DDR4
- **Storage:** Local NVMe SSD with 250 GB capacity; used for both raw documents and Qdrant vector index
- **Operating System:** Windows 11 Enterprise

8.2 Software Stack

The system is implemented primarily in Python. Key components include:

- **Python:** Version 3.13.6 (64-bit)
- **LangChain:** v0.3.x (core, community, huggingface, qdrant integrations)
- **Sentence-Transformers / HuggingFace:** `all-MiniLM-L6-v2` for embedding generation
- **Qdrant:** Python client v1.15.1 with local storage backend
- **llama.cpp:** Accessed via `llama_cpp_python` v0.3.15, using quantized GGUF model files

- **Supporting libraries:** `numpy`, `scikit-learn`, `scipy`, `torch`, `tqdm`, `regex` (for analysis, ML utilities, and runtime efficiency)

8.3 Dataset

The dataset consists of 16 meeting minutes collected from three sources: Teams Copilot exports, Confluence pages, and locally stored text files. All documents were processed through the pipeline described in Chapter 5.

To measure performance, the following testing queries were established:

Table 8.1: Testing prompts used for evaluating retrieval and LLM performance.

Index	Testing Query
1	What was discussed about e net testing in the dailies?
2	What was discussed on the 30/07/25?
3	What was discussed on the 07/08/25?
4	Summarize what was discussed in the Brownbag Sessions this week.
5	Summarize what was discussed in the Sprint Planning this week.

8.4 Metrics & Instruments

Evaluation metrics were designed to cover retrieval quality, answer quality, and efficiency.

Retrieval Quality

- **Recall@5:** Measures how many relevant documents are included in the top-k results.

- **nDCG@5:** Normalized Discounted Cumulative Gain at 5, evaluates ranking quality by rewarding relevant documents that appear higher in the result list.

Answer Quality

- **Human evaluation:** Answer quality was evaluated based on two aspects: the subjective usefulness of the LLM responses (*Satisfactory* vs. *Unsatisfactory*) and the measured LLM output time.

Efficiency

Efficiency was evaluated based on retrieval and LLM response time.

9 Results

9.1 Retrieval & RAG Performance

Retrieval & RAG performance evaluation is recorded in Tables 9.1 and 9.2. The system was tested with the same query once with relevant filters and once without filters. For confidentiality reasons, the actual generated outputs are not included in this report, since they were based on real project meeting minutes (see Section 3.1, Privacy requirement).

Table 9.1: Retrieval and LLM performance without filters.

Index	Query	Recall@5	nDCG@5	Retrieval Time (s)	LLM Output	LLM Output Time (m)
1	What was discussed about e net testing in the dailies	1.00	1.00	0.021	Satisfactory	1:05
2	What was discussed on the 30/07/25	1.00	0.3869	0.022	Unsatisfactory	2:00
3	What was discussed on the 07/08/25	0	0	0.024	Unsatisfactory	1:12
4	Summarize what was discussed in the Brownbag Sessions this week	1.00	1.00	0.023	Satisfactory	1:50
5	Summarize what was discussed in the Sprint Planning this week	1.0	1.00	0.024	Satisfactory	2:03

Table 9.2: Retrieval and LLM performance with metadata filters.

Index	Query	Filter Type	Filter Value	Recall@5	nDCG@5	Retrieval Time (s)	LLM Output	LLM Output Time (s)
1	What was discussed about e net testing in the dailies	Date	01/07/2025 – 01/09/2025	1.00	1.00	0.021	Satisfactory	2:08
2	What was discussed on the 30/07/25	Date	30/07/2025 – 30/07/2025	1.00	1.00	0.023	Satisfactory	1:27
3	What was discussed on the 07/08/25	Date	07/08/2025 – 07/08/2025	1.00	1.00	0.020	Satisfactory	1:53
4	Summarize what was discussed in the Brownbag Sessions this week	Meeting Type	Brownbag Session	1.00	1.00	0.027	Satisfactory	1:04
5	Summarize what was discussed in the Sprint Planning this week	Meeting Type	Sprint Planning	1.00	1.00	0.023	Satisfactory	2:01

The results indicate that semantic search is effective at retrieving topically relevant documents, but its performance varies depending on query type. However, without metadata filters, performance was mixed: while broad topic queries (e.g., “e-net testing” or “Brownbag sessions”) achieved perfect Recall@5 and nDCG@5, date-specific queries performed poorly, with low nDCG@5 and sometimes no relevant documents in the

top-5. Applying strict metadata filters consistently improved retrieval performance to perfect scores.

9.2 Answer Quality

Without filters, the LLM struggled with date-specific queries: although some relevant documents were present in the top-5, low nDCG@5 meant that the correct context was not ranked first, leading to unsatisfactory answers (e.g., for the query on 07/08/25, no relevant document was retrieved). In contrast, broader domain-specific questions such as "What was discussed about e-net testing in the dailies?" consistently yielded satisfactory results, as the relevant document was retrieved at rank 1 and passed into the LLM. With metadata filters applied, answer quality improved across all tested queries, producing satisfactory outputs.

While retrieval itself remained very fast (0.02–0.03s), LLM output times were substantially slower, typically between one and two minutes on a local setup. The delay was primarily due to inference speed limitations of the local model rather than retrieval performance.

The LLM produced accurate and contextually useful answers when the most relevant document was ranked first, since only the top result was passed to the model due to context window limitations. However, when the correct document was missing or buried in the ranking, the answers became vague or included unsupported information.

9.3 Case Studies and Error Analysis

Two representative queries illustrate the system's behavior under different conditions and reveal the main error categories observed.

Success Case

> **Query:** (Table index 1) "What was discussed about e-net testing in the dailies?" **Retrieved Answer:** Satisfactory **Outcome:** The relevant document was

ranked first, ensuring the LLM received the correct context and produced a precise response.

Failure Case

Query: (Table index 3) "What was discussed on the 07/08/25?" **Retrieved Answer:** No relevant documents were included in the top-5 results. **Outcome:** The LLM produced an unsatisfactory response due to lack of context, underscoring the sensitivity of retrieval performance to query phrasing and metadata coverage.

From these cases, two error categories were identified:

- **Hallucinations:** When no relevant context was retrieved, the LLM occasionally generated plausible but unsupported statements.
- **Missed documents:** Date-specific queries were often poorly captured without filters, resulting in low Recall@5 and nDCG@5.

Applying strict metadata filters resolved most of these issues by ensuring that the correct documents were retrieved and passed into the LLM.

10 Discussion

The evaluation highlights the complementary roles of semantic search and metadata filtering. Semantic embeddings are effective at capturing topical relevance, which explains their strong performance on broader queries. However, they are not sufficient on their own for handling precise requests such as specific dates or meeting types. Metadata filters add this missing precision, ensuring that retrieval aligns with the exact constraints defined by the user. In practice, this shows that hybrid strategies—semantic search combined with structured filtering—are necessary for enterprise use cases where both breadth and exactness matter.

Another important observation concerns performance trade-offs. Retrieval was consistently fast, with negligible differences between filtered and unfiltered runs, underscoring the efficiency of the vector database. By contrast, the main bottleneck was LLM inference, which often required one to two minutes per query on local hardware. This latency is a significant barrier to practical adoption and points to the need for hardware acceleration or more efficient model architectures in production environments.

The error analysis also underscores the dependence of answer quality on retrieval robustness. When relevant documents were ranked too low, they were excluded from the context window, which caused vague or unsupported answers. Larger context windows, enabled by more performant hardware, could mitigate this limitation. In parallel, improved prompt design may help the LLM extract more accurate answers even when partial context is available.

Taken together, these findings suggest that successful RAG deployments require both technical optimizations—faster inference, larger context windows—and thoughtful system design that balances semantic flexibility with structured constraints.

11 Conclusion & Future Work

11.1 Conclusion

The goal of this project was to design and evaluate a RAG system for accessing knowledge from meeting minutes, guided by the requirements of project managers, engineers, and consultants. The system successfully met the defined success criteria: meeting minutes were ingested and vectorized into a local Qdrant database, and the LLM generated context-aware summaries when provided with the correct documents. The solution also fulfilled the practical requirement of on-premises deployment, ensuring compliance with data privacy constraints.

The evaluation results confirmed the primary hypothesis (**H1**) that a RAG-based approach can deliver accurate, semantically relevant answers. In particular, broad queries about topics such as testing or specific meeting types consistently achieved perfect retrieval scores and led to satisfactory answers. The secondary hypothesis (**H2**) was also validated: applying metadata filters substantially increased precision for narrow queries, especially those involving specific dates, which semantic similarity alone struggled to resolve.

However, limitations were also observed. The system's effectiveness was constrained by the limited context window of the local LLM, which prevented relevant but lower-ranked documents from being passed into the model. This occasionally resulted in vague or unsupported answers. Additionally, while retrieval was consistently fast, generation latency remained high (often one to two minutes), reflecting the limits of running a large LLM on local hardware. Overall, despite these challenges, the project can be considered a success: it demonstrates that a local, privacy-preserving RAG pipeline is technically feasible and capable of producing useful, reliable answers in a real-world enterprise setting.

11.2 Future Work

Several directions for future improvement were identified:

- **Auto metadata generation:** In this project, curating metadata was a time-consuming manual task, and the overhead will only increase as the dataset grows. While feasible at the prototype stage, future work should explore automated metadata generation to ensure scalability without sacrificing retrieval precision.
- **Scaling to larger datasets:** The system should be tested with extensive collections of meeting minutes spanning many months to evaluate scalability and retrieval robustness in more demanding scenarios.
- **User evaluation:** Real-world testing with project managers, engineers, and consultants would provide qualitative insights into usability, trust, and perceived value.
- **Hardware optimization:** Deploying the LLM on more powerful GPUs or exploring more efficient model architectures could significantly reduce generation latency and enable larger context windows.
- **Query chaining:** Extending the interface to allow users to build on previous queries and answers would support more conversational, exploratory knowledge discovery.
- **LLM-driven query generation:** An agent-style approach, where the LLM autonomously formulates additional sub-queries to the vector database, could further improve recall and ensure that the most relevant context is always retrieved.

In summary, the implemented system provides a solid foundation for intelligent, semantically driven retrieval of meeting minutes. With further optimization and user-centered enhancements, it has strong potential to evolve into a practical tool that significantly improves knowledge access and decision-making in enterprise environments.

Figures

4.1 Document Vectorizer: Documents & Metadata → Chunking → Embedding → Indexing → Vector Database 13
4.2 RAG Query Pipeline: Query → Pre-Filtering → Vector Database Retrieval → Relevant Documents → LLM → Response 14

Tables

3.1 Requirements Traceability Matrix: Linking stakeholders to functional requirements. 10

5.1 Metadata schema for meeting minutes chunks. 16

8.1 Testing prompts used for evaluating retrieval and LLM performance. . . 28

9.1 Retrieval and LLM performance without filters. 30
9.2 Retrieval and LLM performance with metadata filters. 30

Listings

6.1 Qdrant collection setup in DocVectorizer.py 19
6.2 Linking metadata to Qdrant payload and filtering at retrieval 20

7.1 Defining the retriever in Python 22
7.2 Applying metadata filters in RagQuery.py 23
7.3 Initializing and integrating the LlamaCpp model 26

References

Abbreviations

LLM	Large Language Model
RAG	Retrieval-Augmented Generation
Qdrant	Qdrant Vector Database
LlamaCpp	Llama.cpp Library for running LLaMA models locally
nDCG	Normalized Discounted Cumulative Gain
IR	Information Retrieval

Glossary

Best Matching 25

A ranking function used by traditional keyword-based search engines. BM25 estimates the relevance of documents based on term frequency, inverse document frequency, and document length, widely applied in classic IR[1]..

Confluence

An enterprise collaboration and documentation platform developed by Atlassian. It is commonly used to store meeting notes, project documentation, and shared knowledge within organizations..

Embedding

A numerical representation of data (such as text, images, or audio) in a continuous vector space. Embeddings capture semantic relationships, allowing similar items to be located near each other and enabling tasks like semantic search, clustering, and machine learning..

Information Retrieval

A field of computer science that focuses on searching, retrieving, and ranking relevant information from large collections of data. Information Retrieval underpins technologies such as search engines, recommender systems, and enterprise search solutions, using models ranging from keyword-based methods (TF–IDF, BM25) to modern semantic retrieval with embeddings and vector databases..

Metadata

Data that provides information about other data, such as structure, context, or meaning, rather than the content itself. In the context of information systems, metadata helps organize, interpret, and manage data more effectively..

[1] Information Retrieval

Term Frequency–Inverse Document Frequency

A statistical measure used to evaluate the importance of a word in a document relative to a collection or corpus. TF–IDF is the basis for many early keyword-based retrieval systems..

Vector Database

A specialized database designed to store, index, and query high-dimensional vector embeddings. It enables efficient similarity search and retrieval operations, which are essential for applications such as semantic search, recommendation systems, and LLMs that rely on vector representations of data..